Introduction to XML

Mark William Guinto

What is XML?

XML (Extensible Markup Language) is a markup language similar to HTML, but without predefined tags to use. Instead, you define your own tags designed specifically for your needs. This is a powerful way to store data in a format that can be stored, searched, and shared. Most importantly, since the fundamental format of XML is standardized, if you share or transmit XML across systems or platforms, either locally or over the internet, the recipient can still parse the data due to the standardized XML syntax.

There are many languages based on XML, including XHTML, MathML, SVG, RSS, and RDF. You can also define your own.

Structure of an XML document

The whole structure of XML and XML-based languages is built on tags.

XML declaration

XML - declaration is not a tag. It is used for the transmission of the meta-data of a document.

html
Copy to Clipboard
```
<?xml version="1.0" encoding="UTF-8"?>
```

Displaying XML

XML is usually used for descriptive purposes, but there are ways to display XML data. If you don't define a specific way for the XML to be rendered, the raw XML is displayed in the browser.

One way to style XML output is to specify CSS to apply to the document using the `xml-stylesheet` processing instruction.

The Difference Between XML and HTML

XML and HTML were designed with different goals:

- XML was designed to carry data - with focus on what data is
- HTML was designed to display data - with focus on how data looks
- XML tags are not predefined like HTML tags are

The Difference Between XML and HTML

XML and HTML were designed with different goals:

- XML was designed to carry data - with focus on what data is
- HTML was designed to display data - with focus on how data looks
- XML tags are not predefined like HTML tags are

XML Simplifies Things

- XML simplifies data sharing
- XML simplifies data transport
- XML simplifies platform changes
- XML simplifies data availability

Many computer systems contain data in incompatible formats. Exchanging data between incompatible systems (or upgraded systems) is a time-consuming task for web developers. Large amounts of data must be converted, and incompatible data is often lost.

XML stores data in plain text format. This provides a software- and hardware-independent way of storing, transporting, and sharing data.

XML also makes it easier to expand or upgrade to new operating systems, new applications, or new browsers, without losing data.

With XML, data can be available to all kinds of "reading machines" like people, computers, voice machines, news feeds, etc.

Example of XML File

```xml
<?xml version="1.0" encoding="UTF-8"?>
<library>
  <book id="bk101">
    <title>XML Developer's Guide</title>
    <author>Gambardella, Matthew</author>
    <genre>Computer</genre>
    <price>44.95</price>
    <publish_date>2000-10-01</publish_date>
    <description>An
 in-depth look at creating applications
      with XML.</description>
  </book>
  <book id="bk102">
    <title>Midnight Rain</title>
    <author>Ralls, Kim</author>
    <genre>Fantasy</genre>
    <price>5.95</price>
    <publish_date>2000-12-16</publish_date>
    <description>A former
 architect battles corporate zombies,
      an evil sorceress, and her own childhood to become queen
      of the world.</description>
  </book>

</library>
```

This XML example represents a library with two books.

Here's a breakdown:

- `<?xml version="1.0" encoding="UTF-8"?>`: This line declares the XML version and character encoding used in the document.
- `<library>`: This is the root element, encompassing all the information about the library.
- `<book>`: This element represents a single book in the library. It has attributes like `id` to uniquely identify each book.
- `<title>`, `<author>`, `<genre>`, `<price>`, `<publish_date>`, `<description>`: These are child elements of `<book>` providing details about the book.

This structure demonstrates how XML uses tags to organize and label data in a hierarchical way, making it both human-readable and machine-readable.

Sources and related content

END

Introduction to Python

Mark William Guinto

What is Python

Python is a widely used general-purpose, high-level programming language. It was created by Guido van Rossum in 1991 and further developed by the Python Software Foundation. It was designed with an emphasis on code readability, and its syntax allows programmers to express their concepts in fewer lines of code. Python is a programming language that lets you work quickly and integrate systems more efficiently

Python and Java

Hello world in Python and Java

By tradition, Hello World (https://openstax.org/r/100helloworld) is the first program to write when learning a new language. This program simply displays the message "Hello, World!" to the user. The hello world program is only one line in Python:

```
print("Hello, World!")
```

In contrast, the hello world program is five lines in Java (a different language).

```
public class Hello {
  public static void main(String[] args) {
   System.out.println("Hello, World!");
  }
}
```

Python is a versatile language used in a wide range of applications. Here are some of the most common and practical uses:

1. Web Development:

- **Backend Frameworks:** Python offers robust frameworks like Django and Flask, which simplify web development tasks, enabling rapid prototyping and efficient development of scalable web applications.
- **APIs and Web Services:** Python is excellent for building RESTful APIs, allowing different applications to communicate with each other.

2. Data Science and Machine Learning:

- **Data Analysis and Visualization:** Libraries like Pandas and NumPy provide powerful tools for data manipulation, cleaning, and analysis. Matplotlib and Seaborn are used for creating insightful visualizations.
- **Machine Learning:** Python is the go-to language for machine learning, with libraries like scikit-learn offering a wide range of algorithms for tasks like classification, regression, and clustering. TensorFlow and PyTorch are popular for deep learning.

Python is a versatile language used in a wide range of applications. Here are some of the most common and practical uses:

3. Automation and Scripting:

- **Task Automation:** Python can automate repetitive tasks, such as file management, system administration, and web scraping, saving time and increasing efficiency.
- **Scripting:** Python can be used to write scripts for various purposes, like automating software testing, interacting with databases, and managing network devices.

4. Desktop Applications:

- **GUI Development:** Python provides libraries like Tkinter and PyQt for creating graphical user interfaces for desktop applications.

5. Education and Teaching:

- **Beginner-Friendly:** Python's simple syntax and readability make it an ideal language for beginners to learn programming concepts.
- **Versatile Applications:** Python can be used to teach various computer science concepts, from basic programming to advanced topics like data structures and algorithms.

Python is a versatile language used in a wide range of applications. Here are some of the most common and practical uses:

6. Other Notable Uses:

- **Game Development:** Libraries like Pygame simplify game development.
- **DevOps and System Administration:** Python is used for automating server deployments, managing infrastructure, and monitoring systems.
- **Scientific Computing:** Python is used in fields like bioinformatics, physics, and mathematics for numerical analysis and simulations.

These are just a few examples of the many practical uses of Python. Its versatility, extensive libraries, and active community make it a powerful tool for a wide range of tasks.

Basic output

The **`print()`** function displays output to the user. **Output** is the information or result produced by a program. The `sep` and `end` options can be used to customize the output. Table 1.1 shows examples of `sep` and `end`.

Multiple values, separated by commas, can be printed in the same statement. By default, each value is separated by a space character in the output. The `sep` option can be used to change this behavior.

By default, the `print()` function adds a newline character at the end of the output. A **newline** character tells the display to move to the next line. The `end` option can be used to continue printing on the same line.

Code	Output
`print("Today is Monday.")` `print("I like string beans.")`	Today is Monday. I like string beans.
`print("Today", "is", "Monday")` `print("Today", "is", "Monday", sep="...")`	Today is Monday Today...is...Monday
`print("Today is Monday, ", end="")` `print("I like string beans.")`	Today is Monday, I like string beans.
`print("Today", "is", "Monday", sep="? ", end="!!")` `print("I like string beans.")`	Today? is? Monday!!I like string beans.

Python - Lesson 8

Mark William Guinto

Variables

Python has no command for declaring a variable.

A variable is created the moment you first assign a value to it.

```
x = 5

y = "John"

print(x)

print(y)
```

Variables

Variables do not need to be declared with any particular *type*, and can even change type after they have been set.

```
x = 4        # x is of type int

x = "Sally" # x is now of type str

print(x)
```

Casting

If you want to specify the data type of a variable, this can be done with casting.

```
x = str(3)     # x will be '3'

y = int(3)     # y will be 3

z = float(3)   # z will be 3.0
```

Single or Double Quotes?

If you want to specify the data type of a variable, this can be done with casting.

```
x = "John"

# is the same as

x = 'John'
```

Case-Sensitive

```
a = 4

A = "Sally"

#A will not overwrite a
```

Variable Names

A variable can have a short name (like x and y) or a more descriptive name (age, carname, total_volume).
Rules for Python variables:

- A variable name must start with a letter or the underscore character
- A variable name cannot start with a number
- A variable name can only contain alpha-numeric characters and underscores (A-z, 0-9, and _)
- Variable names are case-sensitive (age, Age and AGE are three different variables)
- A variable name cannot be any of the Python keywords.

```
myvar = "John"

my_var = "John"

_my_var = "John"

myVar = "John"

MYVAR = "John"

myvar2 = "John"
```

illegal Variable Names

```
2myvar = "John"

my-var = "John"

my var = "John"
```

Remember that variable names are case-sensitive

ILLEGAL Variable Names

```
2myvar = "John"

my-var = "John"

my var = "John"
```

Remember that variable names are case-sensitive

Python Variables - Assign Multiple Values

```
x, y, z = "Orange", "Banana", "Cherry"

print(x)

print(y)

print(z)
```

Global Variables

Variables that are created outside of a function (as in all of the examples in the previous pages) are known as global variables.

Global variables can be used by everyone, both inside of functions and outside.

Create a variable outside of a function, and use it inside the function

```
x = "awesome"


def myfunc():
  print("Python is " + x)


myfunc()
```

If you create a variable with the same name inside a function, this variable will be local, and can only be used inside the function. The global variable with the same name will remain as it was, global and with the original value.

Create a variable inside a function, with the same name as the global variable

```
x = "awesome"


def myfunc():

  x = "fantastic"

  print("Python is " + x)


myfunc()


print("Python is " + x)
```

In programming, data type is an important concept.

Variables can store data of different types, and different types can do different things.

Python has the following data types built-in by default, in these categories:

Text Type:	`str`
Numeric Types:	`int`, `float`, `complex`
Sequence Types:	`list`, `tuple`, `range`
Mapping Type:	`dict`
Set Types:	`set`, `frozenset`
Boolean Type:	`bool`
Binary Types:	`bytes`, `bytearray`, `memoryview`
None Type:	`NoneType`

Setting the Specific Data Type

If you want to specify the data type, you can use the following constructor functions:

Example	Data Type
`x = str("Hello World")`	str
`x = int(20)`	int
`x = float(20.5)`	float
`x = complex(1j)`	complex
`x = list(("apple", "banana", "cherry"))`	list
`x = tuple(("apple", "banana", "cherry"))`	tuple
`x = range(6)`	range
`x = dict(name="John", age=36)`	dict
`x = set(("apple", "banana", "cherry"))`	set
`x = frozenset(("apple", "banana", "cherry"))`	frozenset
`x = bool(5)`	bool
`x = bytes(5)`	bytes
`x = bytearray(5)`	bytearray

Python Modules

Consider a module to be the same as a code library.

A file containing a set of functions you want to include in your application.

Create a Module

To create a module just save the code you want in a file with the file extension `.py` :

Example

Save this code in a file named `mymodule.py`

```
def greeting(name):
  print("Hello, " + name)
```

Use a Module

Now we can use the module we just created, by using the `import` statement:

Example

Import the module named mymodule, and call the greeting function:

```
import mymodule

mymodule.greeting("Jonathan")
```

String Concatenation

To concatenate, or combine, two strings you can use the + operator.

```
a = "Hello"
b = "World"
c = a + b
print(c)
```

To add a space between them, add a `" "`:

```
a = "Hello"
b = "World"
c = a + " " + b
print(c)
```

Python Conditions and If statements

Python supports the usual logical conditions from mathematics:

- Equals: `a == b`
- Not Equals: `a != b`
- Less than: `a < b`
- Less than or equal to: `a <= b`
- Greater than: `a > b`
- Greater than or equal to: `a >= b`

These conditions can be used in several ways, most commonly in "if statements" and loops.

An "if statement" is written by using the `if` keyword.

```
a = 33
b = 200
if b > a:
  print("b is greater than a")
```

Indentation

Python relies on indentation (whitespace at the beginning of a line) to define scope in the code. Other programming languages often use curly-brackets for this purpose.

```
a = 33
b = 200
if b > a:
print("b is greater than a") # you will get an error
```

Elif

The `elif` keyword is Python's way of saying "if the previous conditions were not true, then try this condition".

```
a = 33

b = 33

if b > a:

  print("b is greater than a")

elif a == b:

  print("a and b are equal")
```

Else

The `else` keyword catches anything which isn't caught by the preceding conditions.

```
a = 200
b = 33
if b > a:
  print("b is greater than a")
elif a == b:
  print("a and b are equal")
else:
  print("a is greater than b")
```

Else

The `else` keyword catches anything which isn't caught by the preceding conditions.

```
a = 200
b = 33
if b > a:
  print("b is greater than a")
elif a == b:
  print("a and b are equal")
else:
  print("a is greater than b")
```